

CardioIA — Fase 3 — Relatório Parte 2 (Nuvem e visualização)

Disciplina / projeto: CardioIA — IoT, transmissão para nuvem e dashboard em tempo real

Escopo: Fluxo de comunicação **MQTT** (ESP32 → broker → Node-RED) e **configuração do dashboard** (gráfico, gauge, indicador de alerta)

Referências no repositório: `esp32/src/main.cpp`, `esp32/src/secrets.h.example` (apenas estrutura — credenciais reais ficam em `secrets.h` local), `esp32/readme.md`, export de referência em `dashboard/dashboard-node-red.json`

1. Objetivo deste relatório

Atender ao entregável da **Parte 2** do enunciado: descrever o **fluxo de comunicação MQTT** entre o dispositivo edge e a nuvem, o papel do **broker** e a forma como o **Node-RED** consome as mensagens para montar um **dashboard** com gráfico, gauge e alerta por limiar — incluindo decisões práticas de configuração (TLS, tópico, parse de JSON e widgets).

2. Visão geral do fluxo ponta a ponta

O protótipo segue o padrão **publicar/assinar (pub/sub)**:

1. O **ESP32** (firmware em C++, Wokwi ou placa física) atua como **publisher**: quando a lógica de “rede simulada” e o transporte permitem, ele envia **uma mensagem JSON por amostra** (telemetria) para um **tópico MQTT** definido no projeto.
2. O **HiveMQ Cloud** atua como **broker**: recebe as publicações, mantém a sessão dos clientes e entrega as mensagens aos **subscribers** que assinaram o tópico (ou um padrão com curinga).
3. O **Node-RED**, em máquina local ou servidor da equipe, usa o nó `mqtt in` como **subscriber**, converte o payload de string para objeto JSON e distribui os campos para os nós de **dashboard** (gráfico, medidor e texto de alerta).

Esse desenho separa bem responsabilidades: o edge **não precisa conhecer** o Node-RED; ele só **publica** em um namespace estável. O dashboard **não precisa rodar** no ESP32; ele **assina** o que interessa ao monitoramento CardioIA.

3. Broker MQTT (HiveMQ Cloud)

3.1 Por que um broker gerenciado

Em ambiente acadêmico, um cluster **HiveMQ Cloud** oferece endpoint público, **TLS** na porta **8883** e **usuário/senha** geridos no painel (Access Management), o que simplifica a demonstração sem manter um servidor MQTT próprio.

3.2 Transporte seguro (TLS)

O firmware usa `WiFiClientSecure` + `PubSubClient`, com conexão ao broker na porta **8883**. No laboratório, costuma-se usar `setInsecure()` para não carregar a cadeia de certificados raiz no sketch — **aceitável para prova de conceito**, com o trade-off de não validar o servidor como em produção. Em uso real, o recomendável é **pinning de CA** ou verificação completa do certificado.

3.3 Autenticação e segredos

Usuário e senha do MQTT são definidos no HiveMQ e copiados para um arquivo local `secrets.h`, **fora do Git** (o repositório mantém apenas `secrets.h.example` como modelo). O Wi-Fi real (fora do Wokwi) segue a mesma ideia: SSID/senha apenas no ambiente local.

3.4 QoS e sessão

As publicações de telemetria em tempo real usam tipicamente **QoS 0** (at most once): menor latência e overhead, adequado a leituras periódicas que se repetem. O Node-RED está configurado com **clean session** e **keepalive** padrão (ex.: 60 s), alinhado ao cliente MQTT do flow.

4. Tópicos e contrato de dados

4.1 Namespace

Adota-se o prefixo `cardioia/f3` e um identificador de dispositivo, por exemplo `esp32-wokwi-01`, para organizar tópicos por projeto e por equipamento:

- **Telemetria:** `cardioia/f3/esp32-wokwi-01/telemetry`

No Node-RED, o nó `mqtt in` pode assinar esse tópico **exato** (como no export de referência) ou um padrão com `+` / `#` se no futuro existirem vários dispositivos (ex.: `cardioia/f3/+/telemetry`).

4.2 Formato JSON (`cardioia.telemetry.v1`)

Cada mensagem é um objeto JSON com campos mínimos para o dashboard e para diagnóstico da fila edge, por exemplo:

- `schema`: versão lógica do payload (`cardioia.telemetry.v1`).
- `device_id`: identificação do emissor.
- `ts_ms`: carimbo em milissegundos (no protótipo, frequentemente `millis()` no ESP32).
- `temp_c`, `hum_pct`: leituras do DHT22; podem ser `null` se a leitura falhar.
- `bpm_sim`: valor derivado dos botões no simulador.
- `wifi_sim`: estado booleano da “rede simulada” no momento da amostra.
- `queue_depth`: profundidade da fila no edge (útil para observar acúmulo offline).
- `batch`: `false` nas publicações unitárias do firmware atual.

O **dreno da fila** no ESP32 publica **várias mensagens** nesse mesmo formato (uma por amostra retirada da fila), o que mantém o Node-RED **simples**: sempre o mesmo parser JSON, sem necessidade de tratar lotes aninhados no protótipo atual.

5. Papel do ESP32 no MQTT (resumo)

Quando `secrets.h` está presente, o firmware tenta associar o ESP32 à rede Wi-Fi (no Wokwi, tipicamente `Wokwi-GUEST` com **IoT Gateway** para alcançar a internet) e, após conectar, estabelece sessão TLS com o HiveMQ. Na fase **online simulada**, a rotina de dreno **retira amostras da fila** e chama `publish` no tópico de telemetria. Se o Wi-Fi real não estiver disponível no simulador, o projeto pode ainda **escrever JSON no Serial** para não perder a demonstração da resiliência — tema mais detalhado no Relatório Parte 1.

6. Node-RED: fluxo lógico

6.1 Entrada e parse

- Nó `mqtt in` (broker configurado com host do cluster, porta **8883**, TLS e credenciais).
- Saída conectada a um nó `json` com ação “**Always convert to JavaScript object**”: o `msg.payload` deixa de ser string e vira objeto, permitindo acesso a `msg.payload.temp_c`, `msg.payload.bpm_sim`, etc.

6.2 Ramificações (fan-out)

Após o `json`, o fluxo **divide** em três caminhos, cada um com um nó `function`:

Ramo	Função	Destino
Temperatura	Extrai <code>temp_c</code> numérico, define <code>msg.topic</code> para legenda da série (ex.: <code>Temp °C</code>)	<code>ui_chart</code> (gráfico de linha)
BPM	Extrai <code>bpm_sim</code> numérico	<code>ui_gauge</code> (medidor 0–180, título “Batimentos por Minuto”)
Alerta	Avalia apenas BPM contra limiares acordados (ex.: normal, >100, <40, opcionalmente >150) e monta texto + <code>msg.class</code> <code>Name</code>	<code>ui_template</code>

Essa separação evita misturar **BPM** e **temperatura** no mesmo eixo do gráfico — decisão comum quando as escalas são muito diferentes.

6.3 Dashboard (node-red-dashboard)

- `ui_tab` / `ui_group` : agrupamento visual “CardioIA”.
- `ui_chart` : série temporal da temperatura.
- `ui_gauge` : visualização do `bpm_sim` (parâmetro “vital” simulado).
- `ui_template` : texto de estado/alerta; como o tema do dashboard pode sobrescrever cores de texto, usa-se `msg.className` no cartão do widget e **CSS com** `!important` no próprio template para cores fiéis (verde / laranja / vermelho conforme a classe).

6.4 TLS no cliente Node-RED

O nó **mqtt-broker** no editor referencia uma configuração **TLS** (`tls-config`), com verificação de certificado desativada no exemplo de laboratório (`verifyservercert: false`), espelhando a pragmática do firmware. Em produção, recomenda-se validar o servidor.

7. Regras de alerta no dashboard (exemplo da equipe)

No protótipo atual, o **indicador textual** concentra-se no **BPM simulado** (sem alerta de temperatura no mesmo bloco, por escolha da equipe). Exemplos de limiar:

- Faixa considerada **ok** para a mensagem verde.
- **BPM alto** acima de um limiar (ex.: 100).
- **BPM baixo** abaixo de 40 (com cuidado para não alertar quando o valor é zero ou inválido).
- Opcional: faixa **crítica** acima de 150 com classe de cor distinta.

Esses valores são **educacionais** e devem ser explicitados no vídeo/relato da entrega como **não clínicos**.

8. Eficiência, segurança e dados pessoais (reflexão breve)

- **Eficiência:** publicar **um JSON por amostra** simplifica o Node-RED; o custo é mais tráfego MQTT quando a fila drena após offline — aceitável no escopo do curso.
 - **Segurança:** TLS + credenciais fora do repositório; evitar prints públicos com senhas; rotacionar credenciais se houver vazamento em vídeo ou screenshot.
 - **LGPD / ética:** dados do protótipo são **fictícios**; em IoT de saúde real seriam necessários consentimento, minimização de dados, controle de acesso ao broker e trilha de auditoria — tópicos a mencionar no README ou em aula.
-

9. Conclusão

A Parte 2 do CardioIA implementa o encadeamento pedagógico **transmissão** → **broker** → **visualização**: o ESP32 publica telemetria estruturada em JSON no HiveMQ Cloud (TLS), e o Node-RED assina o tópico, parseia o payload e alimenta **gráfico** (temperatura), **gauge** (BPM simulado) e **alerta** por texto com cores estáveis via `className` e CSS. O desenho é extensível para mais dispositivos (wildcard no tópico) e para políticas de QoS/retained diferentes, documentando-se sempre o contrato JSON e os limiares no dashboard.